Zero-Dependency MQTT Broker

Project Proposal - IT365 Advanced Computer Networks

Pranav Bhat (231IT049)
Pranav Vinodh (231IT050)
Rijul Raman (231IT056)

National Institute of Technology Karnataka, Surathkal

Introduction

- **MQTT (Message Queuing Telemetry Transport):** The standard publish-subscribe messaging protocol for IoT and edge devices.
- **The Standard Approach:** Most deployments rely on heavy, generalized third-party brokers (e.g., Mosquitto, HiveMQ).
- **The Core Concept:** A lightweight, custom-built MQTT 3.1.1 broker developed entirely from scratch using native OS socket primitives without external networking or protocol libraries.
- **Focus Area:** High-performance, low-level binary packet parsing and stateful protocol management over TCP.

Literature Survey

- **OASIS Standard (MQTT Version 3.1.1):** Defines the strict binary packet structures, variable-length headers, and Quality of Service (QoS) delivery requirements.
- **Trie-Based Topic Routing (H. Jiang et al.):** Research highlights that simple hash maps fail for wildcard subscriptions (+, #). Radix trees (Tries) are the optimal data structure for O(K) topic resolution.
- **C10K Problem & Concurrent IO (D. Kegel):** Standard blocking sockets fail at scale. Literature on event-driven I/O (`epoll`/`select`) provides the foundation for managing hundreds of concurrent broker connections.

Problem Statement

The Challenge

Modern IoT infrastructures often deploy overly complex brokers that carry unnecessary dependencies. Understanding the inner workings of an application-layer network protocol requires implementing its binary parsing and state management natively.

Key Technical Hurdles:

- Decoding variable-length, bit-shifted binary headers directly from a TCP stream.
- Efficiently routing publish payloads to subscribers using wildcards.
- Managing stateful client sessions (Clean Session vs. Persistent) when TCP connections drop.

Objectives

1. **Native TCP Server:** Develop a non-blocking TCP server capable of handling concurrent client connections without third-party networking libraries.
2. **Protocol Parser:** Implement a strict byte-level decoder compliant with MQTT 3.1.1 to process `CONNECT`, `PUBLISH`, `SUBSCRIBE`, and `PINGREQ` packets.
3. **Routing Engine:** Construct a Radix Tree (Trie) data structure to handle dynamic topic matching and wildcard resolution.
4. **Session Management:** Build a thread-safe registry to track client connectivity states, keep-alive timers, and subscription maps.

Methodology

- **Phase 1: Foundation**
  - Initialize the non-blocking TCP server socket architecture.
  - Implement the bitwise packet decoder for fixed and variable length headers.
- **Phase 2: Handshake & Routing**
  - Handle `CONNECT` / `CONNACK` state logic.
  - Build the Topic Trie for `SUBSCRIBE` resolution.
- **Phase 3: Payload Fan-out & Resilience**
  - Traverse the Trie on `PUBLISH` and fan-out binary payloads to client sockets.
  - Implement `PINGREQ` keep-alive timeouts and handle unexpected socket disconnections.

Conclusion

- By the end of this project, we will have a fully functional, zero-dependency MQTT broker.
- This implementation will demonstrate deep proficiency in application-layer protocol design.
- The project transitions networking theory into practice by solving real-world challenges in bitwise parsing, concurrent socket management, and algorithmic routing.

References


Banks, A., Gupta, R.
MQTT Version 3.1.1. OASIS Standard, 2014.


Stevens, W. R., Fenner, B., Rudoff, A. M.
UNIX Network Programming: The Sockets Networking API. Addison-Wesley Professional, 3rd Edition, 2003.


Morrison, D. R.
PATRICIA - Practical Algorithm To Retrieve Information Coded in Alphanumeric. Journal of the ACM (JACM), 1968.

Thank You

Questions?